

UI Design Choice for CI/CD Pipeline with Automated Failure Recovery

CS 331 — Software Engineering
UI Selection, Justification and Comparison

1. Overview of UI Types Considered

Before selecting a UI style, four candidate interface types were evaluated against the requirements of this project. Each type has different strengths depending on the user, task complexity, and frequency of use.

UI Type	How It Works	Best For
Command Language Interface (CLI)	User types text commands. System parses and executes them.	Expert users, scripting, automation, repetitive tasks
Menu-Based Interface	User selects options from a fixed list of menus or dropdowns.	Novice users, limited option sets, data entry forms
Direct Manipulation Interface (DMI)	User interacts with visual representations of objects — clicking, dragging, real-time feedback.	Dashboards, monitoring tools, complex state visualization
Form-Fill Interface	User fills structured forms with fields, checkboxes, dropdowns.	Data entry, configuration, structured one-time tasks

2. Chosen Interface Type: Direct Manipulation Interface (DMI)

The chosen interface type for this project is a **Direct Manipulation Interface (DMI)** implemented as a React-based web dashboard running on port 3000.

A Direct Manipulation Interface works by giving the user visual representations of system objects — pipelines, service health indicators, log analysis results, failure severity badges — that they can interact with directly. Actions have immediate visible feedback. State is shown in real time. The user does not need to remember commands or navigate through menus to find what they need.

3. Justification

3.1 The Users and Their Needs

The primary users of this system are developers and DevOps engineers who need to monitor pipeline health, diagnose failures, and verify that automated recovery is working. These users:

- Monitor multiple pipelines simultaneously during deployments
- Need to spot failures immediately without reading raw JSON or terminal output
- Understand visual indicators like red/green status, severity badges, and progress states
- Occasionally need to trigger pipelines or test notifications manually
- Are not always experts in every service — they should not need to memorise API endpoints

A Direct Manipulation Interface serves all of these needs better than any alternative.

3.2 Why Direct Manipulation Fits This Project

Project Requirement	How DMI Satisfies It
5 microservices running simultaneously	All service statuses shown as live colour-coded cards on the dashboard. Green = healthy, red = down. Status refreshes every 15 seconds automatically.
Need to see pipeline success vs failure at a glance	Each pipeline event is shown as a row in a table with a colour-coded badge (SUCCESS in green, FAILED in red). No scrolling through logs required.
Log analysis is complex — needs structured presentation	The Log Analyzer page shows failure category, severity, confidence score, and individual findings as distinct visual elements, not raw JSON.
Recovery actions vary (retry, rollback, restart, manual)	The Failure Classifier result page shows the recovery action in large bold text with colour coding (ROLLBACK in red, RETRY in cyan) so it stands out immediately.
Non-technical stakeholders may view the dashboard	Visual icons, colour coding, and clear labels mean someone without deep technical knowledge can understand the system state without training.
Triggering pipelines manually during testing	A dedicated Trigger Pipeline page with a form, auto-generated event IDs, and real-time job status polling handles this through point-and-click interaction.

3.3 Specific DMI Principles Applied

The implementation follows the three core principles of Direct Manipulation defined by Ben Shneiderman (1983):

Principle 1 — Continuous representation of objects of interest

All five microservices (Pipeline Controller, Log Analyzer, Failure Classifier, Recovery Manager, Notification Service) are shown as persistent cards on the dashboard at all times. The user always knows which services are running without issuing any command.

Principle 2 — Physical actions instead of complex syntax

To trigger a pipeline event, the user fills a form and clicks a button. Internally this calls POST /pipeline-event with a JSON payload, but the user never sees or writes that. To analyze a log, the user pastes text and clicks Analyze Log. The API call happens invisibly.

Principle 3 — Rapid, incremental, reversible operations with immediate feedback

After clicking Trigger Pipeline, the UI immediately shows the API response and begins polling the job status every two seconds, updating the display as it changes. After clicking Analyze Log, results appear in milliseconds with categorised findings. The user does not wait for a batch output or parse terminal text.

4. Why the Other UI Types Were Rejected

Command Language Interface (CLI) — Rejected

A CLI would require developers to memorise endpoint URLs, JSON field names, and exact syntax for every operation.

Example of what triggering a pipeline would look like in CLI:

```
curl -X POST http://localhost:9000/pipeline-event -H "Content-Type: application/json"
-d '{"event_id":"evt-001","pipeline_id":"my-app","run_number":1,"status":"FAILED","log_url":"http://..."}'
```

This is error-prone, provides no visual feedback, and requires significant training.

CLIs are excellent for automation and scripting, but this system already has automation built in.

The human-facing interface should focus on visibility and monitoring, not command memorisation.

Menu-Based Interface — Rejected

Menu-based interfaces force the user to navigate a tree of options to reach any action.

In a monitoring context this is too slow — if a pipeline fails at 2am the developer needs to see the problem instantly, not navigate: Main Menu → Services → Pipeline → Status → View.

Menu interfaces also do not support real-time updating — menus are static.

They work well for configuration and settings but not for live monitoring dashboards.

Form-Fill Interface — Partially Used, Not Primary

Form-fill interfaces were used as a secondary component within the DMI — specifically on the Trigger Pipeline page and the Send Notification page where structured input is needed.

However, a purely form-fill interface for the whole system would mean no live monitoring, no real-time service health, and no visual log analysis results.

Forms are appropriate for discrete tasks; they are not suitable as the primary interaction model for a monitoring and recovery system.

5. UI Structure and Pages

The chosen DMI is divided into four pages, each corresponding to a distinct user task:

Page	Primary Interaction Style	Purpose
Dashboard	Direct manipulation — live status cards refresh automatically	Monitor all 5 services and view recent pipeline event history
Trigger Pipeline	Form-fill within DMI — with immediate response feedback and live polling	Manually fire pipeline events for testing and demonstration
Log Analyzer	Direct manipulation — click sample logs, click Analyze, click Classify	Paste and analyze Jenkins build logs, view failure classification
Notifications	Form-fill within DMI — with channel delivery confirmation	Send manual alerts and browse notification history

6. Design Decisions That Support the DMI Choice

Colour coding throughout

Green is consistently used for success and healthy states. Red for failure and critical severity. Yellow for warnings and degraded states. Cyan for active/informational states. This is consistent with established conventions in monitoring tools like Grafana, Datadog, and AWS CloudWatch, meaning users with any DevOps background can read the dashboard immediately.

No page refresh required

Service health polling runs on a 15-second interval using JavaScript `setInterval`. Job status polling runs every 2 seconds after triggering a pipeline. The user never needs to press F5 to see updated state — this is a core property of direct manipulation systems.

Sample data for demonstration

The Log Analyzer page provides four pre-written sample logs (Build Error, Test Failure, Deploy Error, Dependency Error) as one-click buttons. This supports demonstration and testing without requiring a live Jenkins instance, which is appropriate for a college project demo environment.

Persistent history

Recent pipeline events and notification history are stored in browser `localStorage`. This means the history survives page refresh and provides context for the current state of the system without requiring a separate database service.

Sidebar navigation

A fixed left sidebar shows all pages at all times. The active page is highlighted. This is a direct manipulation pattern — the user can see all possible contexts and switch instantly with one click, rather than using a back button or a dropdown menu.

7. Comparison Summary

Criterion	CLI	Menu-Based	Form-Fill	DMI (Chosen)
Learning curve	High	Low	Low	Low
Speed for expert users	High	Low	Medium	High
Real-time monitoring	No	No	No	Yes
Visual failure recognition	No	Partial	No	Yes
Non-technical user access	No	Yes	Yes	Yes
Immediate feedback	Partial	No	Partial	Yes
Suitable for this project	No	No	Partial	Yes

8. Conclusion

The Direct Manipulation Interface was chosen for the CI/CD Pipeline Dashboard because it best matches the nature of the system and the needs of its users. The system involves multiple concurrent services, real-time state changes, and failure events that require immediate visibility. A command language interface would require memorisation and provide no visual monitoring. A menu-based interface would be too slow for monitoring tasks. A pure form-fill interface has no real-time capability.

The DMI provides continuous visible representation of all five microservices, immediate feedback on every user action, and colour-coded visual language that allows rapid failure recognition. Form-fill components are used within the DMI specifically where structured input is needed — for triggering pipelines and sending notifications — which is the correct and limited role for form-fill interaction.

This choice aligns with how professional monitoring tools in the industry — Grafana, Datadog, Jenkins Blue Ocean, GitHub Actions — approach the same problem: real-time visual dashboards with direct manipulation, not command-line text output.